

The pi-agent port, certified in zero knowledge

Machine-checked in Lean 4 with Mathlib
RequestProject/PiPortZK/

Abstract

The entire pi-agent port — 929 files of the ingested pi-agent corpus, 6 181 055 bytes of source, 1 634 190 tokens, 2 443 emitted Lean definitions, 6 543 816 bytes of Lean, 929 of 929 files reproducing their source byte for byte — checked against a declaration it makes about itself, arithmetised into 930 rank-1 constraints, committed to, and argued for in zero knowledge. Published digest: 179 (mod 10007). Everything in this paper is printed by the Lean definitions the theorems are about, and no step of the argument uses an axiom beyond Lean’s own `propext`, `Classical.choice` and `Quot.sound`.

1 The object: the pi-agent port

The pi-agent port is the tree under `ported/`. The rule engine that produces it is itself written in Lean (`RequestProject/PiPort.lean`, driven by `RequestProject/PiPortRun.lean`): elaborating the project reads the ingested corpus dump, applies the bootstrap rules to every file, checks the lossless round trip at build time, and writes the ported Lean tree together with `ported/PORT_MANIFEST.txt`. `scripts/piportzk.py` re-measures that port out of the working tree — the manifest for the per-file source size, token count and emitted definition count, and the emitted Lean files themselves for their size on disk — and generates the catalogue `RequestProject/PiPortZK/Data.lean`. Run with `--check` it fails if the table has drifted from the tree, so the argument below cannot be about a port that is no longer there.

One catalogue entry is:

```
structure PiFile where
  /-- the file's path inside the pi-agent corpus -/
  path : String
  /-- the Horner code of the path (`PiPortZK.strCode path`) -/
  pathCode : ℕ
  /-- `0` executable source, `1` documentation, `2` configuration and the rest -/
  kind : ℕ
  /-- size of the source file in bytes -/
  bytes : ℕ
  /-- number of tokens the port's lexer produced for it -/
  tokens : ℕ
  /-- number of Lean definitions emitted for it -/
  defs : ℕ
  /-- size in bytes of the emitted Lean file, measured on disk -/
  outBytes : ℕ
  /-- whether the emitted Lean file reproduces the original bytes exactly -/
  lossless : Bool
  deriving DecidableEq, Repr
```

kind	files	source bytes	tokens	definitions	digest
executable source	778	5 463 512	1 451 257	2 389	3955
documentation	79	512 047	132 422	17	6285
configuration and the rest	72	205 496	50 511	37	996
all	929	6 181 055	1 634 190	2 443	179

The totals are theorems, not annotations: the kernel computes them from the table.

```
theorem piPort_totals :
  piBytes = 6181055 ∧ piTokens = 1634190 ∧ piDefs = 2443 ∧ piOutBytes = 6543816 := by
  refine ⟨by rfl, by rfl, by rfl, by rfl⟩

theorem kind_sizes :
  (kindFiles 0).length = 778 ∧ (kindFiles 1).length = 79 ∧ (kindFiles 2).length = 72 := by
  refine ⟨by rfl, by rfl, by rfl⟩
```

2 The self-declaration

The template of `RequestProject/Merged/` asks of a program that it declare all the data needed to check it, checks the program against its own declaration, arithmetises that check, and proves knowledge of a satisfying witness in zero knowledge. Here the program is the port. Its declaration is a piece of data:

```
structure PiClaim where
  /-- if set, every ported file must round-trip losslessly -/
  requireLossless : Bool
  /-- every file belongs to a kind with index at most this -/
  maxKind : ℕ
  /-- every source file has at least this many bytes -/
  minBytes : ℕ
  /-- the lexer produced at least this many tokens for every file -/
  minTokens : ℕ
  deriving DecidableEq, Repr
```

```
def piClaim : PiClaim :=
  { requireLossless := true, maxKind := 2, minBytes := 1, minTokens := 1 }
```

So the port claims of each of its 929 files: that the emitted Lean reproduces the source byte for byte (`requireLossless := true`); that the file is executable source, documentation or configuration (`maxKind := 2`); that it is non-empty and produced at least one token; that it produced no more tokens than it has bytes and no more definitions than tokens; and that the Lean file it was rendered into is at least as large as the source it carries. The semantic reading and the port's own Boolean check are separate definitions, proved equivalent, so the claim is decidable and can be arithmetised:

```
def PiFile.Meets (f : PiFile) (c : PiClaim) : Prop :=
  (c.requireLossless = true → f.lossless = true) ∧ f.kind ≤ c.maxKind ∧
  c.minBytes ≤ f.bytes ∧ c.minTokens ≤ f.tokens ∧ f.tokens ≤ f.bytes ∧
  f.defs ≤ f.tokens ∧ f.bytes ≤ f.outBytes

def PiFile.eval (f : PiFile) (c : PiClaim) : Bool :=
  (!c.requireLossless || f.lossless) && decide (f.kind ≤ c.maxKind) &&
  decide (c.minBytes ≤ f.bytes) && decide (c.minTokens ≤ f.tokens) &&
  decide (f.tokens ≤ f.bytes) && decide (f.defs ≤ f.tokens) &&
  decide (f.bytes ≤ f.outBytes)

theorem PiFile.eval_iff (f : PiFile) (c : PiClaim) :
  f.eval c = true ↔ f.Meets c := by
  cases hc : c.requireLossless <;> cases hl : f.lossless <;>
  simp [PiFile.eval, PiFile.Meets, hc, hl, and_assoc]
```

Lifting the check to the whole catalogue, the indices the checker accepts are found by *running* the checker, and all of them are accepted exactly when the port is sound:

```
theorem PiCatalogue.acceptedIdx_length_eq_iff (cat : PiCatalogue) (c : PiClaim) :
  (cat.acceptedIdx c).length = cat.size ↔ cat.Sound c := by
  have h := List.length_filter_eq_length_iff (p := cat.accepts c) (l := List.range cat.size)
  rw [List.length_range] at h
  rw [PiCatalogue.acceptedIdx, h]
  simp only [List.mem_range]
  constructor
  · intro h f hf
    obtain ⟨i, hi⟩ := List.mem_iff_getElem?.mp hf
    have hlt : i < cat.size := by
      by_contra hge
      rw [List.getElem?.eq_none (by simp [PiCatalogue.size] using Nat.le_of_not_lt hge)] at hi
      exact absurd hi (by simp)
    have := h i hlt
    rw [PiCatalogue.accepts, hi] at this
    simp [PiFile.eval_iff] using this
  · intro h i hi
    rw [PiCatalogue.accepts, List.getElem?.eq_getElem hi]
    simp [PiFile.eval_iff] using h _ (List.getElem_mem hi)
```

3 The port meets its declaration

This is the kernel running the port’s own checker over every one of the 929 entries:

```
theorem piPort_sound : piCatalogue.Sound piClaim := by decide
```

```
theorem piPort_all_lossless : piLossless = 929 := by rfl
```

The digest is the public handle on the table: a Horner fold over the entries, reduced modulo 10007. Nothing about it is claimed to be collision-resistant — it is a concrete stand-in for a hash, as the fixed linear hash below is a stand-in for a random oracle — but it is computed by the kernel from the real measurements, and the zero-knowledge argument is an argument about it.

```
def fileDigest (f : PiFile) : ℕ :=
  ((((((f.pathCode * 31 + f.kind) * 31 + f.bytes) * 31 + f.tokens) * 31 + f.defs) * 31 +
    f.outBytes) * 31 + (if f.lossless then 1 else 0)) % modulus
```

```
theorem piPort_digests :
  piDigest = 179 ∧ kindDigest 0 = 3955 ∧ kindDigest 1 = 6285 ∧ kindDigest 2 = 996 := by
  refine ⟨by rfl, by rfl, by rfl, by rfl⟩
```

The path of a file enters its digest through the Horner code recorded in the table; folding 929 string literals inside the kernel costs minutes, so the code is measured alongside the byte counts and spot-checked against the paths themselves:

```
theorem pathCode_sample : ∀ f ∈ piFiles.take 8, strCode f.path = f.pathCode := by decide
```

4 Arithmetisation

The port claims something universal about itself — *every* file it produced meets the declaration — so the constraint system is the universal one: one booleanity constraint per file, and one counting constraint saying that the accepted cells sum to 929. That is 930 rank-1 constraints over $\mathbb{F}_{10007}$, of which 929 cells are accepted.

```
def piR1CS (cat : PiCatalogue) (c : PiClaim) : R1CS F :=
  boolConstraints cat.size ++
  [eqConstConstraint (cellsLC (cat.acceptedIdx c)) (cat.size : F)]
```

Over a field with more residues than the port has files, n bits can sum to n only if there are n of them and every one is 1. Hence the arithmetisation is faithful in both directions — it is satisfiable exactly when the port meets its declaration, so a satisfying witness is evidence and not decoration:

```
theorem piR1CS_satisfiable_iff (cat : PiCatalogue) (c : PiClaim) (hq : cat.size < q) :
  (∃ w : ℕ → ZMod q, w 0 = 1 ∧ R1CS.Sat w (piR1CS cat c)) ↔ cat.Sound c := by
  constructor
  · rintro ⟨w, h0, hsat⟩
    exact sound_of_piR1CS_sat cat c hq w h0 hsat
  · intro h
    exact ⟨allOnes, rfl, piR1CS_sat_allOnes cat c h⟩
```

5 The certificate

Over $\mathbb{F}_{10007}$ the prover commits to the port’s digest vector — the digest of the whole measured table together with the digests of its three kinds — blinded by fresh randomness, with the public commitment key

$$A = \begin{pmatrix} 3 & 1 & 4 & 1 \\ 5 & 9 & 2 & 6 \end{pmatrix}, \quad \Phi(v, \rho) = Av + \rho.$$

```
def piVec : Fin 4 → K := ![179, 3955, 6285, 996]
```

That the committed numbers really are the digests of the measured table is itself a theorem:

```
theorem piVec_eq_digests :
  piVec = ![(piDigest : ℕ) : K), ((kindDigest 0 : ℕ) : K), ((kindDigest 1 : ℕ) : K),
  ((kindDigest 2 : ℕ) : K)] := by
  obtain ⟨h0, h1, h2, h3⟩ := piPort_digests
  rw [h0, h1, h2, h3]
  norm_num [piVec]
```

Applying the Fiat–Shamir transform with a fixed linear stand-in for the random oracle gives the published data:

published item	value
committed digest vector (secret opening)	(179, 3955, 6285, 996)
commitment $y = \Phi(w)$	(3748, 920)
first message $a = \Phi(k)$	(27, 131)
challenge $c = H(y, a)$	7480
response, value part	(7991, 2715, 8922, 4880)
response, blinding part	(8253, 5485)

The verification equation $\Phi(z) = a + cy$ holds by kernel computation, and the published data are exactly the honest prover's output:

```
theorem piCert_verifies :
  FSVerify (comMap phi psi) hash statement firstMessage response := by
  show comMap phi psi response = firstMessage + challenge • statement
  decide

theorem piCert_data :
  statement = ![3748, 920] ∧ firstMessage = ![27, 131] ∧ challenge = 7480 ∧
  response.1 = ![7991, 2715, 8922, 4880] ∧ response.2 = ![8253, 5485] := by
  refine ⟨by decide, by decide, by decide, by decide, by decide⟩
```

Two accepting certificates on one first message under different hash values yield an opening of the published commitment — the knowledge property; and the joint law of the commitment and the whole interaction does not depend on which digest vector was committed to — the zero-knowledge property:

```
theorem piCert_extraction (H H' : (Fin 2 → K) → (Fin 2 → K) → K)
  (z z' : (Fin 4 → K) × (Fin 2 → K))
  (h : FSVerify (comMap phi psi) H statement firstMessage z)
  (h' : FSVerify (comMap phi psi) H' statement firstMessage z')
  (hne : H statement firstMessage ≠ H' statement firstMessage) :
  comMap phi psi
    (extract (H statement firstMessage) (H' statement firstMessage) z z') = statement :=
  fs_extraction (comMap phi psi) H H' statement firstMessage z z' h h' hne
```

```
theorem piCert_hides_port (v v' : Fin 4 → K) (e : K) :
  jointView phi psi v e = jointView phi psi v' e :=
  joint_view_indep_of_witness phi psi psi_surjective v v' e
```

6 Everything in one statement

The layers are collected into a single theorem: faithful arithmetisation, the port's soundness, the lossless round trip of every file, the identification of the committed vector with the measured digests, verification, honesty of the transcript, knowledge, and zero knowledge.

```
theorem piPort_end_to_end :
  ((∃ w : ℕ → Certificate.K, w 0 = 1 ∧
    R1CS.Sat w (piR1CS piCatalogue piClaim)) ↔ piCatalogue.Sound piClaim) ∧
  piCatalogue.Sound piClaim ∧
  piLossless = 929 ∧
  Certificate.piVec = ![(piDigest : ℕ) : Certificate.K], ((kindDigest 0 : ℕ) : Certificate.K),
  ((kindDigest 1 : ℕ) : Certificate.K), ((kindDigest 2 : ℕ) : Certificate.K)] ∧
  FSVerify (comMap Certificate.phi Certificate.psi) Certificate.hash
    Certificate.statement Certificate.firstMessage Certificate.response ∧
  (Certificate.firstMessage, Certificate.challenge, Certificate.response) =
  prover (comMap Certificate.phi Certificate.psi) Certificate.witness
    Certificate.randomness Certificate.challenge ∧
  (∀ (H H' : (Fin 2 → Certificate.K) → (Fin 2 → Certificate.K) → Certificate.K)
    (z z' : (Fin 4 → Certificate.K) × (Fin 2 → Certificate.K)),
    FSVerify (comMap Certificate.phi Certificate.psi) H Certificate.statement
      Certificate.firstMessage z →
    FSVerify (comMap Certificate.phi Certificate.psi) H' Certificate.statement
      Certificate.firstMessage z' →
    H Certificate.statement Certificate.firstMessage ≠
      H' Certificate.statement Certificate.firstMessage →
    comMap Certificate.phi Certificate.psi
```

```

      (extract (H Certificate.statement Certificate.firstMessage)
        (H' Certificate.statement Certificate.firstMessage) z z') =
    Certificate.statement) ∧
  (∀ (v v' : Fin 4 → Certificate.K) (e : Certificate.K),
    jointView Certificate.phi Certificate.psi v e =
      jointView Certificate.phi Certificate.psi v' e) :=
  (Certificate.pi_r1cs_satisfiable_iff_sound,
    piPort_sound,
    piPort_all_lossless,
    Certificate.piVec_eq_digests,
    Certificate.piCert_verifies,
    Certificate.piCert_eq_honest,
    fun H H' z z' h h' hne => Certificate.piCert_extraction H H' z z' h h' hne,
    Certificate.piCert_hides_port)

```

7 Do we have the proof? The axiom audit

A Lean proof is only as good as what it rests on. `RequestProject/PiPortZK/Audit.lean` prints the transitive axiom closure of every headline result; this is that output, verbatim. No `sorry` occurs anywhere in the layer, and nothing beyond Lean’s three standard axioms is used — in particular no compiled evaluation (`Lean.ofReduceBool`), so every computation above was done by the kernel itself.

```

'PiPortZK.PiFile.eval_iff' depends on axioms: [propext]
'PiPortZK.PiCatalogue.eval_iff' depends on axioms: [propext, Quot.sound]
'PiPortZK.PiCatalogue.acceptedIdx_length_eq_iff' depends on axioms: [propext, Quot.sound]
'PiPortZK.piPort_sound' depends on axioms: [propext, Quot.sound]
'PiPortZK.piPort_all_lossless' does not depend on any axioms
'PiPortZK.piPort_totals' depends on axioms: [propext]
'PiPortZK.kind_sizes' does not depend on any axioms
'PiPortZK.pathCode_sample' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.piPort_digests' does not depend on any axioms
'PiPortZK.piR1CS_satisfiable_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.Certificate.piVec_eq_digests' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.Certificate.piCert_verifies' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.Certificate.piCert_data' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.Certificate.piCert_extraction' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.Certificate.piCert_hides_port' depends on axioms: [propext, Classical.choice, Quot.sound]
'PiPortZK.piPort_end_to_end' depends on axioms: [propext, Classical.choice, Quot.sound]

```

8 Reproducing this paper

Every number above is printed by `lake exe piportzk`, which runs the same definitions the theorems cite, and every displayed statement is extracted verbatim from the sources:

```

python3 scripts/piportzk.py --check # the table still matches ported/
lake build RequestProject.PiPortZK.Audit
lake build piportzk
./lake/build/bin/piportzk > papers/piport_artifacts.json
python3 scripts/piport_paper.py # writes this PDF

```

A The first rows of the catalogue

The full table is 929 rows of `RequestProject/PiPortZK/Data.lean`; here are the first 30, with the per-entry digests the fold above consumes.

path	kind	bytes	tokens	defs	Lean bytes	digest
README.md	1	5 678	1 552	0	5 783	9590
AGENTS.md	1	10 733	2 740	0	10 836	2199
biome.json	2	812	280	0	916	1472
CONTRIBUTING.md	1	4 825	1 040	0	4 934	4557
LemmaScript-files.txt	1	280	82	0	395	5584
LICENSE	2	1 071	205	0	1 172	1185
package.json	2	2 835	970	0	2 941	7975
pi-test.bat	2	340	87	0	445	3618
pi-test.ps1	2	1 672	340	0	1 777	1555
pi-test.sh	2	1 486	265	0	1 590	2413
README_LemmaScript.md	1	8 325	2 072	3	8 762	4067
SECURITY.md	1	4 229	774	0	4 334	5497
test.sh	2	1 904	271	0	2 005	8526
tsconfig.base.json	2	627	147	0	739	1999
tsconfig.json	2	1 276	542	0	1 383	1194
.npmrc	2	36	12	0	136	2539
packages/agent/README.md	1	15 863	3 779	1	16 209	1790
packages/agent/CHANGELOG.md	1	17 384	5 658	1	17 675	5757
packages/agent/package.json	2	1 384	546	0	1 505	1241
packages/agent/tsconfig.build.json	2	367	181	0	495	725
packages/agent/vitest.config.ts	0	568	169	0	693	7481
packages/agent/vitest.harness.config.ts	0	819	272	0	952	5768
packages/agent/docs/agent-harness.md	1	23 394	4 661	0	23 524	8913
packages/agent/docs/durable-harness.md	1	8 092	1 466	0	8 224	3642
packages/agent/docs/hooks.md	1	10 675	2 623	0	10 799	8039
packages/agent/docs/observability.md	1	8 626	2 080	5	9 270	2282
packages/agent/src/agent-loop.ts	0	20 044	4 172	19	23 097	5822
packages/agent/src/agent.ts	0	17 143	3 846	2	17 575	2408
packages/agent/src/index.ts	0	1 300	336	0	1 421	888
packages/agent/src/node.ts	0	90	28	0	210	7299